BidKV: Request-Level Victim Selection Under KV-Cache Pressure

Advisor-authored storyline draft; implementation and experiments remain student-owned

August 2026

Abstract

When an LLM server exhausts KV-cache capacity, it must choose a request to preempt. Native policies commonly evict from a scheduling queue without explicitly comparing released capacity with recomputation and starvation risk. BidKV is a framework-portable victim-selector interface that ranks requests by released KV tokens per unit of a disruption surrogate. Its canonical vLLM path evicts an entire request and relies on native preempt-and-recompute; its SGLang-aware branch counts only private radix-tree tokens. It does not compress KV tokens or change model semantics. This distinction makes the core hypothesis falsifiable: utility-ranked victim selection should reduce preemption-induced service cost relative to native LIFO, SJF admission plus LIFO, random, and largest-first selection under matched pressure. The repository currently establishes configuration, selector, adapter, audit, and experiment-runner behavior through host tests, but contains no versioned serving result bundle. We therefore present the precise request-level mechanism, oracle, matched experiment, evidence boundary, and stopping rules without claiming latency or throughput improvement.

1 Problem and Motivation

Paged KV memory turns overload into a victim-selection decision. Evicting a large request releases more blocks, but recomputing a nearly finished or repeatedly preempted request may amplify tail latency and starvation. Evicting the newest request may be cheap locally but fail to free enough memory, causing a cascade. The decision should therefore account for both capacity and downstream disruption.

BidKV controls only *which request* is preempted. In vLLM mode A, the framework releases the request's entire current KV footprint and later recomputes it. In the radix-aware SGLang path, shared-prefix state remains resident and only private tokens count as freed capacity. Any description of token compression, selective materialization, or semantic quality degradation would misstate the implemented mechanism.

2 Seven-Question Research Contract

Q1—Problem. Under repeatable KV pressure, which request-level victim rule minimizes service disruption while freeing the required capacity and avoiding repeated preemption?

Q2—Importance. Preemption affects TTFT after recompute, TPOT/ITL, completion rate, queueing, throughput, and fairness. A selector that improves average throughput but causes starvation or tail regressions is not deployable.

Q3—Hidden assumption and related-work boundary. Native eviction often treats queue position, size, or admission order as an adequate proxy for future cost. Cache compression, prefix routing, and tiering instead change representation, placement, or reuse. BidKV contributes only at the request-level victim seam; it must not claim their benefits or compare against them as if they were the same action space.

Q4—Mechanism hypothesis. Rank each eligible victim by

$$U_i = \frac{r_i}{\delta_i + \epsilon}, \qquad \delta_i = 1 + w_c c_i + w_p p_i,$$

where r_i is actually releasable tokens, c_i completion fraction, and p_i prior preemptions. For a radix tree, replace full footprint with private tokens and charge a recompute term proportional to them. This should balance capacity release against late-request and anti-starvation penalties.

Q5—Feasibility. The package provides an inert-by-default native victim-selector entry point, a kill switch, deterministic configuration, framework adapters, decision snapshots, baselines, and an open-loop experiment runner. Host tests show interface behavior; real serving feasibility and measurement fidelity remain unproven in this repository.

Q6—Matched evaluation. Compare native FCFS/LIFO preemption, SJF admission plus LIFO, static random, largest-first, and BidKV using identical runtime commit, model, trace order, arrival process, KV capacity, batching, prefix-cache state, and independent launches. Report candidate universes, selected victims, actual tokens released, recomputation, repeated preemptions, request outcomes, TTFT/TPOT/ITL distributions, throughput, fairness, and selector CPU.

Q7—Takeaway. The intended knowledge is a regime map showing when a capacity/disruption ratio outperforms queue- or size-only victims and when its surrogate fails. A negative result closes the frozen utility formula or a framework-specific branch; it does not justify relabeling the mechanism as compression.

3 Mechanism

3.1 Candidate and action identity

At each pressure event the selector receives the same eligible running-request snapshot used by every arm. A record binds request/generation identity, current computed and prompt tokens, maximum output tokens, prior preemptions, arrival order, private tokens when available, required release, and cache-sharing mode. The action is whole-request eviction through the framework-native path.

The canonical vLLM selector is feature-off by default and falls back to the native scheduling policy unless explicitly enabled and admitted by its pressure gate. Its kill switch has priority. This makes installation inert and creates a matched bypass arm in the same carrier.

3.2 Disruption is a surrogate, not quality

The implementation historically names δ `quality_delta`, but mode A recomputation should preserve output semantics. Here δ is interpreted only as a disruption surrogate: progress already completed plus a penalty for repeated preemption. It is not a measured accuracy or output-quality loss. A paper claim requires calibration showing that the ordering predicts actual recompute/tail cost better than simpler rules.

3.3 Framework-specific released capacity

For vLLM without shared-prefix ownership, r_i is the request's current full KV footprint. For radix-aware SGLang, r_i is private tokens because evicting a request does not release shared prefix state. Mixing these numerators would produce a false cross-framework comparison. Results are therefore reported per branch first, with a common victim-selection abstraction but distinct cache-ownership receipts.

4 Evidence and Claim Boundary

The current repository contains zero-dependency policy code, a native vLLM-HUST selector entry point, legacy adapters, five strategy families, audit/collector logic, and frozen-trace runners. Host tests can establish configuration, sorting, gating, adapter, and serialization semantics. There is no checked-in raw serving result directory, result manifest, model/runtime receipt, or matched end-to-end distribution.

Table 1: Evidence ceiling at this revision.

Evidence	Status and allowed claim
Host unit/integration tests	Complete; selector/API semantics only
Native plugin discovery	Tested contract; not a serving run
Experiment runner	Present; planned protocol only
Matched vLLM results	Missing; no performance claim
Matched SGLang results	Missing; no cross-framework claim
Surrogate calibration	Missing; no predictive-quality claim

The highest evidence is host contract. No smoke, replay, simulation, GPU/NPU probe, real-online traffic, or result is promoted by this paper.

5 Evaluation Contract

The strongest deployable baseline is selected from native LIFO and largest-first on the frozen workload; SJF admission plus LIFO is reported separately because it changes admission as well as victim context. Static random is a sanity baseline. BidKV is the only treatment. All strategies consume the same candidate-universe receipt and must actually free the requested capacity or record a failed pressure event.

Correctness requires 100% request/generation identity, no missing or duplicate completion, token/log-probability equivalence within a preregistered deterministic oracle, cache accounting conservation, and framework invariants after preempt/recompute. Performance is interpreted only after correctness. Before results, freeze a minimum improvement gate over the strongest baseline for preemption-induced tail cost or SLO attainment, plus non-regression gates for throughput, fairness, selector CPU, and failure rate. No threshold is selected from observed treatment data.

Use at least ten independent launches or a complete real event set per cell, counterbalance arm order, and report full distributions. For vLLM-HUST execution, use the official `.23` container and scheduler-provided host/device/model identities; no personal path or fixed accelerator assignment belongs in the prospective contract. CPU-only host tests do not support device claims.

Stop if pressure is too rare, candidate universes differ, actual release disagrees with r_i, output equivalence fails, repeated preemption worsens, or BidKV does not beat the strongest deployable victim baseline after selector overhead. If the positional/disruption surrogate lacks predictive headroom over size-only selection, freeze that negative result and replace the research question rather than tuning weights to the test trace.

6 Threats and Two-Week Delivery

The main internal threat is counterfactual cost: only the chosen victim is recomputed, so alternative-victim disruption cannot be read directly from one run. Matched replay of frozen arrival traces and repeated independent serving runs provide complementary evidence, but replay remains below real serving. Another threat is admission confounding: SJF changes which requests reach the victim set and cannot be called a pure selector baseline.

Days 1–3 freeze versioned workload traces, pressure events, candidate-universe receipts, and correctness oracle. Days 4–7 run native LIFO, largest-first, and BidKV on the smallest real vLLM matrix. Days 8–10 calibrate the disruption surrogate against measured recompute/tail cost and audit starvation. Days 11–14 add SJF and, only if cache ownership is observable, the SGLang radix branch. Student owners retain implementation, environment, experiment, and result responsibility; this draft supplies only the research contract.

7 Related Work Boundary

PagedAttention and vLLM establish block-level KV management and a native serving baseline [1]. A large-provider characterization shows that production KV-cache behavior depends on workload and cache-management effects rather than capacity alone [2]. BidKV is narrower: it selects a request-level preemption victim under a fixed candidate universe. Prefix sharing, compression, and tiering remain separate mechanism classes and cannot be credited to this selector.

8 Conclusion

BidKV is a request-level victim selector, not a KV compressor. Its capacity-over-disruption ranking is plausible and testable, but current evidence stops at host contracts. The next contribution gate is a matched serving result that proves correct native preempt/recompute, calibrates the surrogate, beats the strongest deployable victim baseline, and identifies negative pressure and ownership regimes.

References

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023.

[2] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. In *2025 USENIX Annual Technical Conference*, pages 465–482, 2025.